

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY

Jeffrey J. McConnell

Analysis *of* Algorithms

Second Edition
An Active Learning Approach

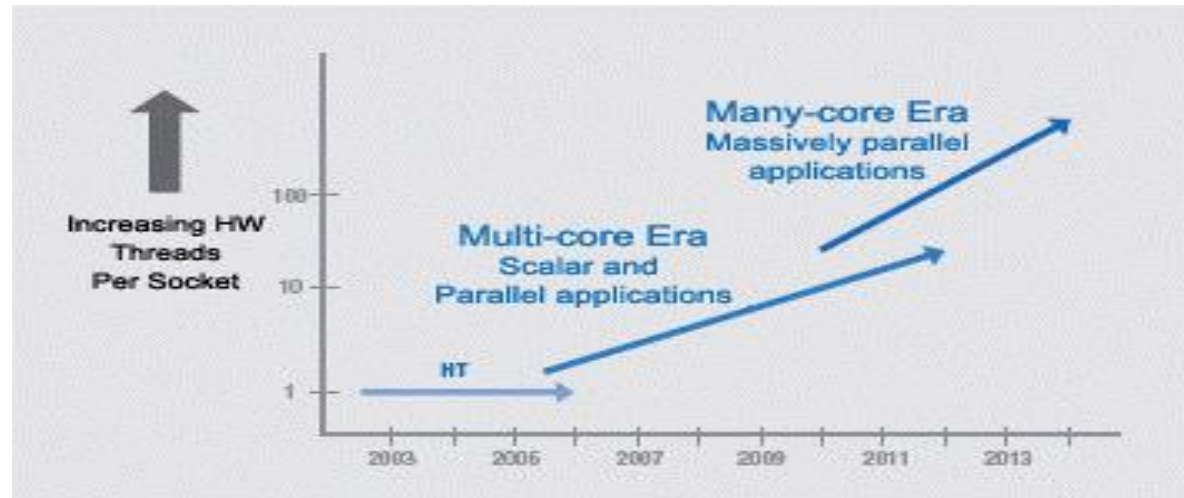


Chapter 9

Parallel Algorithms

Parallelism everywhere

- Warehouse-scale computers, data-centers, clouds
- Supercomputer
- GPUs
- Many-cores



Basic knowledge of parallel algorithms:

- Necessary for these industry platforms

Top500.org's November 2017 ranking

Position	Name	Country	Teraflops	Power (kW)
1	Sunway TaihuLight	China	93,015	15,371
2	Tianhe-2	China	33,863	17,808
3	Piz Daint	Switzerland	19,590	2,272
4	Gyokou	Japan	19,136	1,350
5	Titan	United States	17,590	8,209
6	Sequoia	United States	17,173	7,890
7	Trinity	United States	14,137	3,844
8	Cori	United States	14,015	3,939
9	Oakforest-PACS	Japan	13,555	2,719
10	K Computer	Japan	10,510	12,660

2019??

- <https://www.top500.org/statistics/list/53/countries/>

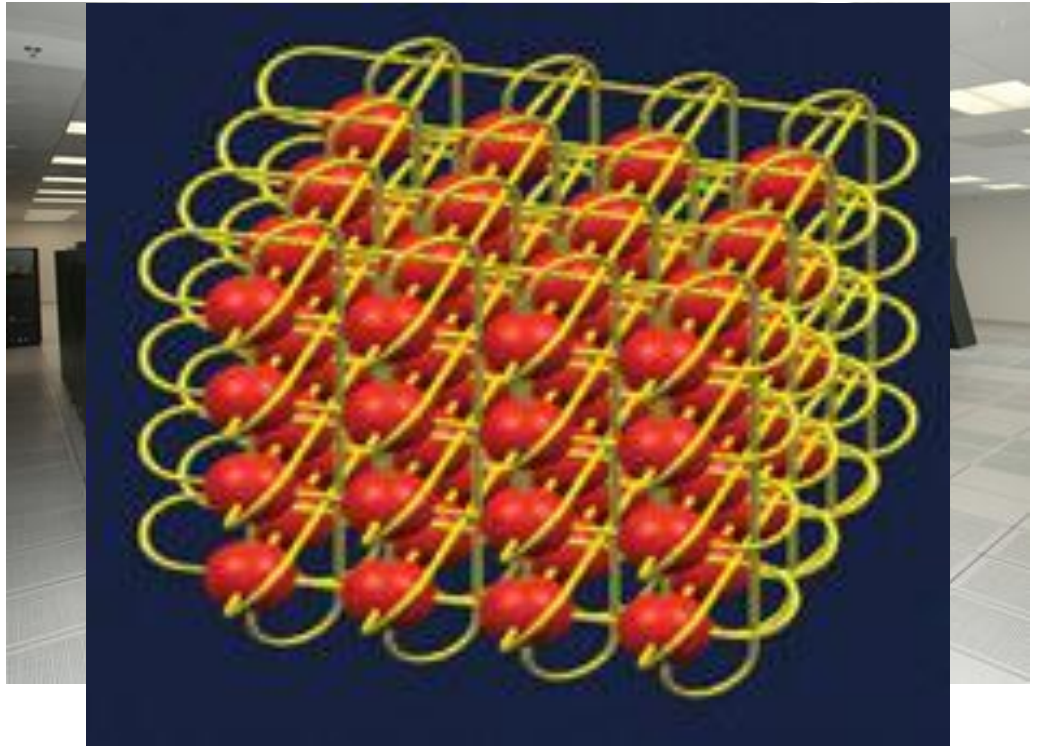
Some Real Systems: IBM BlueGene/L

212,992 CPUs

478 Tflops

At Lawrence
Livermore Nat'l Lab

≈ \$200 Million



3-d toroidal interconnect

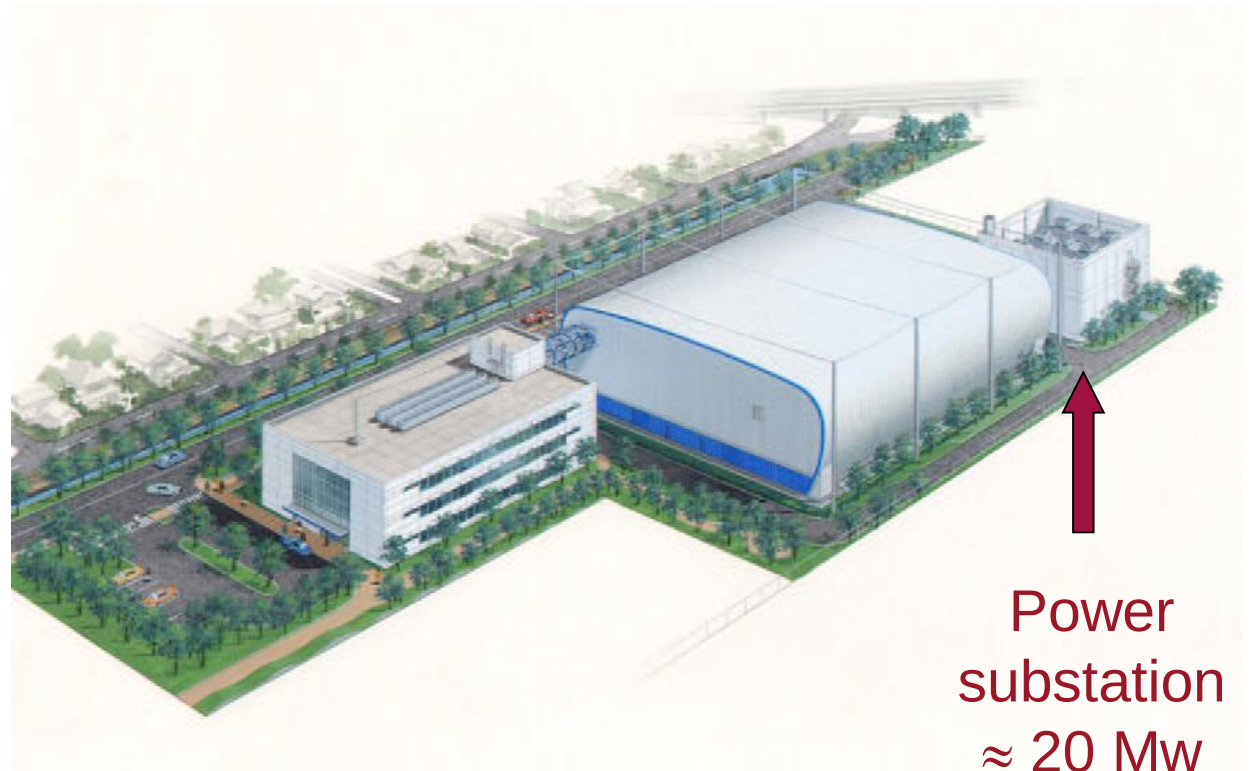
NEC Earth Simulator

5120 Processors

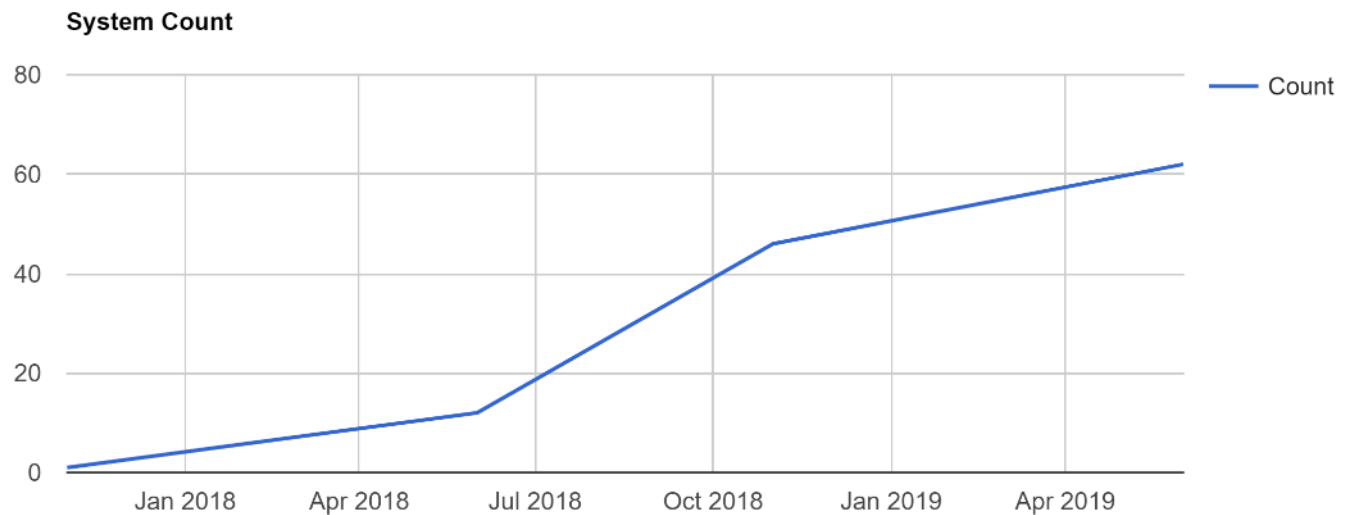
41 Tflops

In Yokohama,
Japan

> \$500 Million



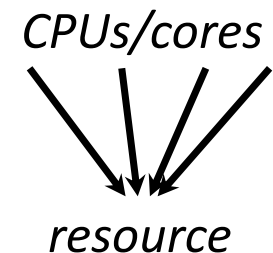
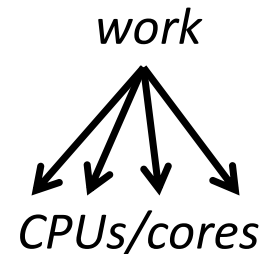
ACCELERATOR/CP FAMILY / NVIDIA VOLTA



List	Count	System Share (%)	Rmax (GFlops)	Rpeak (GFlops)	Cores
Jun 2019	62	12.4	406,316,070	621,405,705	7,481,412
Nov 2018	46	9.2	335,748,290	515,000,540	6,225,940
Jun 2018	12	2.4	224,559,400	360,593,742	4,488,720
Nov 2017	1	0.2	1,070,000	1,819,752	22,440

Parallel vs. concurrent

- **Parallel:** Using multiple processing resources (CPUs, cores) at once to solve a problem faster.
 - Example: A sorting algorithm that has several threads each sort part of the array.
- **Concurrent:** Multiple execution flows (e.g. threads) accessing a shared resource at the same time.
 - Example: Many threads trying to make changes to the same data structure (a global list, map, etc.).
- Many programmers confuse these two concepts.
 - Threads are often used to implement both.



Principles for Parallelism Analysis

- Algorithm speed
 - A count of the operations like in Chapters 2 through 8
- Speed-up
 - How much faster a parallel algorithm is
- Cost
 - The speed of an algorithm multiplied by the number of processors needed
 - For Chapters 2 to 8 the cost and the speed are the same because we used one processor

Speedup in Simplest Terms

$$\text{Speedup} = \frac{\text{Sequential execution time}}{\text{Parallel execution time}}$$

Efficiency

$$\text{Efficiency} = \frac{\text{Sequential running time}}{\text{Processors} \times \text{Parallel running time}}$$

$$\text{Efficiency} = \frac{\text{Speedup}}{\text{Processors}}$$

$$\text{Efficiency} = \frac{\text{Sequential running time}}{\text{Cost}}$$

Efficiency denoted in Quinn by $\varepsilon(n, p)$ for a problem of size n on p processors

Amdahl's Law

Let f be the fraction of operations in a computation that must be performed sequentially, where $0 \leq f \leq 1$. The maximum speedup ψ achievable by a parallel computer with n processors is

$$\psi \equiv S(p) \leq \frac{1}{f + (1-f)/n} \leq \frac{1}{f}$$

- The word “**law**” is often used by computer scientists when it is an observed phenomena (e.g, Moore's Law) and not a theorem that has been proven in a strict sense.
- However, Amdahl's law can be proved for traditional problems

Proof for Traditional Problems: If the fraction of the computation that cannot be divided into concurrent tasks is f , and no overhead incurs when the computation is divided into concurrent parts, the time to perform the computation with n processors is given by $t_p \geq ft_s + [(1 - f)t_s] / n$, as shown below:

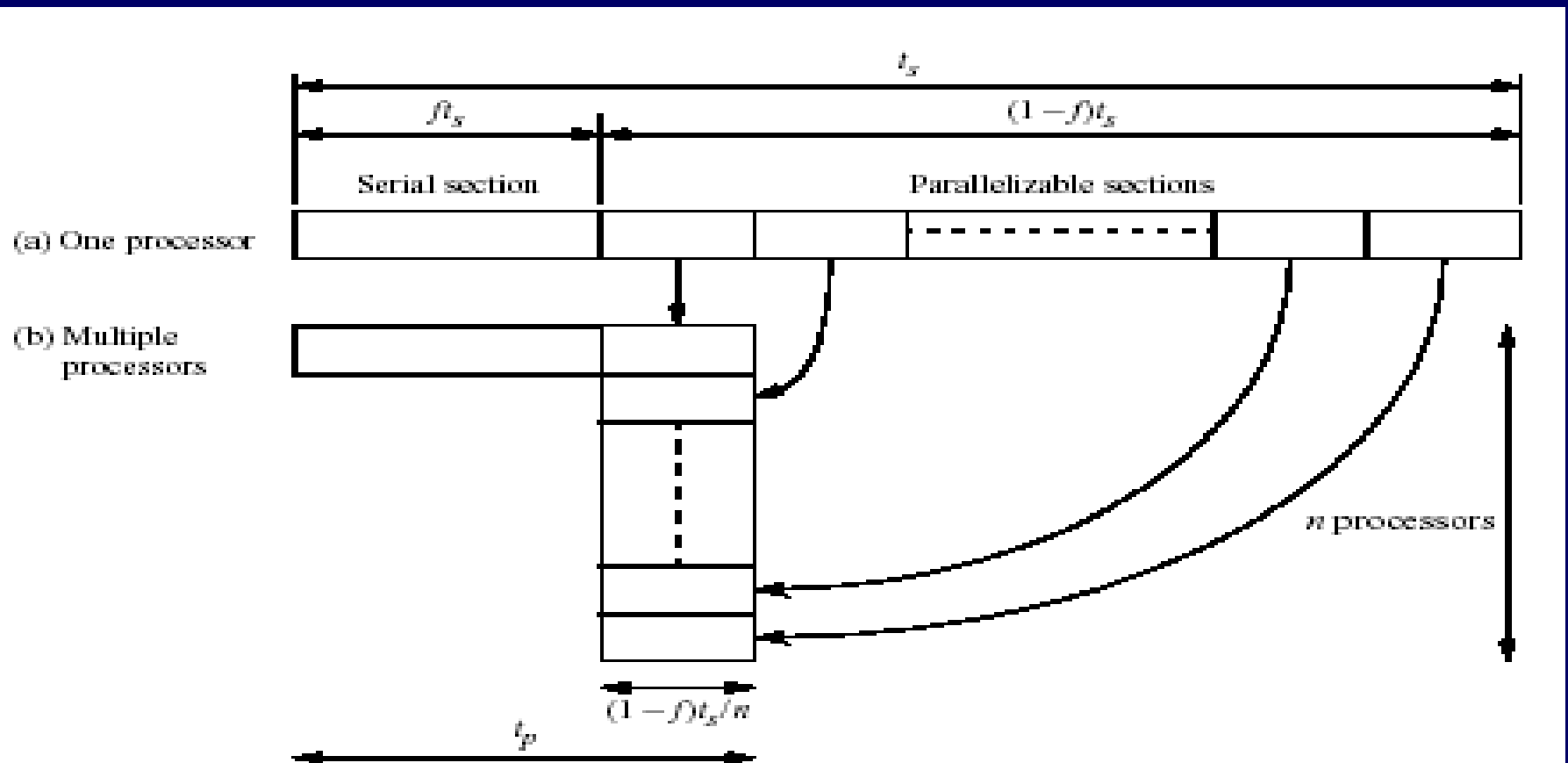


Figure 1.29 Parallelizing sequential problem — Amdahl's law.

Amdahl's Law

- Let $f = 0.1$
- $P = 20$ processors
- What is the upper bound for Speed up?

Amdahl's Law

- Let $f = 0.1$
- $P = 20$ processors
- $S \leq 1 / (0.1 + 0.9/20) = 6.9 \sim 7$

Amdahl's Law

- Let $f = 0.1$
- $P = 100$ processors
- $S \leq 1 / (0.1 + 0.9/100) \sim 9$

Amdahl's Law

- Let $f = 0.1$
- $P = 1000$ processors ??
- Limits ?

Computer System Categories

- Single Instruction Single Data (SISD) (PC)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD) (Pipeline)
- Multiple Instruction Multiple Data (MIMD) (MultiProcessors)

The PRAM Model

- Parallel Random Access Machine
 - Tightly coupled machine
 - Each processor has a small number of registers for temporary storage
 - All processors run on the same cycle:
 - All read data from memory in step 1
 - All process data in step 2
 - All write data to memory in step 3

Memory Contention

- We can have concurrent reads, where more than one processor can read from the same memory location
- We can have exclusive writes, where only one processor can write to one location, while other processors can write to other locations – multiple writes to a location is an error



Memory Contention

- We can have concurrent writes, where more than one processor can write to the same memory location

Write Conflict Resolution

- In concurrent writes, the system will use some mechanism to determine which succeeds:
 - Priority
 - Highest priority processor succeeds
 - Arbitrary
 - Random processor succeeds
 - Common
 - All values being written must be the same
 - Combining
 - Values combined by a predefined operator



Four Types of Memory Access

- Concurrent Read Concurrent Write (CRCW)
- Concurrent Read Exclusive Write (CREW)
- Exclusive Read Concurrent Write (ERCW)
- Exclusive Read Exclusive Write (EREW)

A decorative header with a blue abstract background featuring wavy, liquid-like patterns.

Broadcasting Data in a CREW PRAM Model

- Transfer a data value to all of the other processors
- This process will take two cycles

Broadcasting Data in a CREW PRAM Model Algorithm

P_1 writes the data value into M_1

Parallel Start

for $k = 2$ to p do

P_k reads the data value from M_1

end for

Parallel End

Broadcasting Data in an EREW PRAM Model

- Only one processor can read from a memory location, so each pass, the processors write the data to new locations so that more processors can read it on the next pass

Broadcasting Data in an EREW PRAM Model Algorithm

```
P1 writes the data value into M1
procLoc = 1
for j = 1 to lg p do
  Parallel Start
    for k = procLoc + 1 to 2 * procLoc do
      Pk reads Mk-procLoc
      Pk writes to Mk
    end for k
  Parallel End
  procLoc = procLoc * 2
end for j
```



Analysis

- On pass one 1 processor reads the value, on pass two 2 processors read, on pass three 4 processors read, ...

- 
- Trace the algorithm for:
 - $P = 4$

Analysis

- On pass one 1 processor reads the value, on pass two 2 processors read, on pass three 4 processors read, ...
- Complexity?

Analysis

- On pass one 1 processor reads the value, on pass two 2 processors read, on pass three 4 processors read, ...
- There need to be $O(\lg p)$ passes for the broadcast to complete

Finding the Maximum Value

- Processors compare pairs of values passing on the larger one
- Each pass needs half the processors of the previous one
- Similar to the tournament method that used one processor

Finding the Maximum Value

Algorithm A

```
count = N / 2    (P number of processors)
for i = 1 to (lg count) + 1 do
  Parallel Start
    for j = 1 to count do
      Pj reads M2j-1 into X and M2j into Y
      if X > Y then
        Pj writes X into Mj
      else
        Pj writes Y into Mj
      end if
    end for
  Parallel End
  count = count / 2
end for
```

- 
- Trace the algorithm for:
 - $P = 4$
 - $N = 8$

A decorative header with a blue abstract background featuring wavy, liquid-like patterns.

Analysis

- Complexity?



Analysis

- Each pass creates another level of the tournament tree, so there are $O(\lg N)$ passes

Analysis

- Each pass creates another level of the tournament tree, so there are $O(\lg N)$ passes
- There will be $N/2$ processors needed, giving a cost of $O(N \lg N)$

Analysis

- Each pass creates another level of the tournament tree, so there are $O(\lg N)$ passes
- There will be $N/2$ processors needed, giving a cost of $O(N \lg N)$
- The sequential algorithm had a cost of only $O(N)$, so there must be a better parallel version

Better Version of Finding the Maximum Value

- To get a cost of $O(N)$ with a run-time of $O(\lg N)$, we can only use $N/\lg N$ processors

Better Version of Finding the Maximum Value

- To get a cost of $O(N)$ with a run-time of $O(\lg N)$, we can only use $N/\lg N$ processors
- So, we have a first step in which the processors sequentially find the maximum of $\lg N$ values

Better Version of Finding the Maximum Value

- To get a cost of $O(N)$ with a run-time of $O(\lg N)$, we can only use $N/\lg N$ processors
- So, we have a first step in which the processors sequentially find the maximum of $\lg N$ values
- We then do the original process on the results of this step

Finding the Maximum Value

Algorithm B

Parallel Start

for $j = 1$ to $N/\lg N$ do

P_j finds the maximum of $M_{1+(j-1)*\lg N}$
through $M_{j*\lg(N)}$ using the sequential
algorithm

P_j write the maximum to M_j

end for

Parallel End

count = $(N / \lg N) / 2$

Continues as in Algorithm A

- 
- Trace the algorithm for:
 - $P = 16/4 = 4$
 - $N = 16$

Analysis

- Overall, the algorithm has a time of $O(\lg N)$ and a cost of $O(N)$
- So, the cost is the same but the **time is much faster** than the sequential version